OPTI-CROP PROJECT DELIVERABLE

Coding & Solution Overview

1. Core Backend Implementation

The application logic is hosted inside **`app.py`**. Below are highlights of the model prediction and data processing flows.

ML Classifier Execution

```
# Prepare features vector for prediction
features = np.array([[nitrogen, phosphorous, potassium, temperature, humidity, ph, rainfall]])
prediction = model.predict(features)
predicted_crop = prediction[0]
```

Recommendation Logic & Threshold Auditing

When a crop is recommended, the backend compares input soil metrics against standard ranges (**`crop_ranges.pkl`**) to generate customized feedback:

- If Input Nitrogen < (Mean - Standard Deviation): recommends Nitrogen-rich urea or organic compost.
- If Input pH < 5.5: suggests adding agricultural lime to neutralize acidity.
- If Input pH > 7.5: recommends incorporating organic peat moss or agricultural sulfur.

2. Database Integration

Upon generating a prediction, the app creates three linked SQL records within a single transaction:

1. Inserts soil features in **`soil_data`**.
2. Logs crop prediction, model metadata, and confidence score in **`predictions`**.
3. Stores text advice and result summaries in **`reports`**.

This enforces data integrity and enables historical analysis on the user dashboard.